03 · History

Every reading, newest first, with filters, paging and deleting a reading entered by mistake.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

The History page lists the citizen's readings with filters for one vital, a time window (7, 30, 90, 365 days or all) and "flagged only". It pages 20 at a time. Flagged rows show their severity, the reason and whether a worker has reviewed them. A reading can be deleted after a confirmation.

The filters live in the URL (`?metric=heart_rate&days=30&page=2`), so a filtered view survives a reload.

Files involved

Only the files this feature uses.

```
backend/app/api/routes/vitals.py     GET /health/history, DELETE /health/readings/{id}
frontend/src/
├── pages/citizen/History.tsx         the page
├── lib/params.ts                     oneOf(), intParam(), useQueryState(): safe URL filters
└── services/health.ts                getHistory(), deleteReading()
```

How the pieces connect

1. The page reads the filters from the URL and checks each one (unknown values fall back to defaults).
2. It calls `GET /health/history` with page, page size and filters; the key is the JSON of the query, so any filter change reloads.
3. The backend builds one query, counts the total for paging, and returns one page.
4. Delete asks "Delete this reading?" first, then calls `DELETE /health/readings/{id}` and reloads.

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/api/routes/vitals.py` · history

```
177 @router.get("/history", response_model=HistoryOut)
178 async def history(
179     page: int = Query(1, ge=1, le=10_000),
180     page_size: int = Query(20, ge=1, le=100),
181     metric: str | None = Query(None, max_length=40),
182     days: int | None = Query(None, ge=1, le=365),
183     flagged_only: bool = False,
184     user: User = Depends(require_onboarding_user),
185     db: AsyncSession = Depends(get_session),
186 ) -> HistoryOut:
187     if metric is not None and metric not in VITALS:
188         raise HTTPException(status.HTTP_422_UNPROCESSABLE_ENTITY, f"Unknown metric: {metric}")
189     query = (
190         select(HealthRecord, ProcessedData)
191         .outerjoin(ProcessedData, ProcessedData.health_record_id == HealthRecord.id)
192         .where(HealthRecord.user_id == user.id)
193     )
194     if metric:
195         query = query.where(HealthRecord.vitals[metric].as_string().is_not(None))
196     if days:
197         query = query.where(HealthRecord.timestamp >= now_utc() - timedelta(days=days))
198     if flagged_only:
199         query = query.where(ProcessedData.is_anomaly.is_(True))
200
201     total = await db.scalar(select(func.count()).select_from(query.subquery())) or 0
202     rows = await db.execute(
203         query.order_by(HealthRecord.timestamp.desc(), HealthRecord.id).offset((page - 1) * page_size).limit(
                page_size)
```

```
    )
    return HistoryOut(
        items=[_reading_out(record, processed) for record, processed in rows.all()],
        total=total,
        page=page,
        page_size=page_size,
    )
```

What the code means

- Each filter is a typed parameter: `page: int = Query(1, ge=1, le=10_000)` must be 1 to 10,000 (`10_000` is just 10000 written readably).
- The query is built step by step: `query = query.where(...)` adds a condition only if that filter was used.
- `HealthRecord.vitals[metric].as_string().is_not(None)` looks inside the JSON column: "this reading includes that vital".
- `select(func.count()).select_from(query.subquery())` counts all matching rows for the page numbers.
- `.offset((page - 1) * page_size).limit(page_size)` skips earlier pages and takes one page.

2. `backend/app/api/routes/vitals.py` · delete_reading

```
@router.delete("/readings/{reading_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_reading(
    reading_id: uuid.UUID,
    request: Request,
    user: User = Depends(require_onboarding_user),
    db: AsyncSession = Depends(get_session),
) -> Response:
    # Scoped to the caller: someone else's reading is simply "not found".
    record = await db.scalar(
        select(HealthRecord).where(HealthRecord.id == reading_id, HealthRecord.user_id == user.id)
    )
    if record is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, READING_NOT_FOUND)
    # The score goes with it; a visit booked from it keeps going (case_id is SET NULL).
    await db.execute(delete(ProcessedData).where(ProcessedData.health_record_id == record.id))
    await db.delete(record)
    db.add(
        AuditLog(
            user_id=user.id,
            action="delete_reading",
            entity="health_records",
            entity_id=record.id,
            ip_address=client_ip(request) or "unknown",
        )
    )
    await db.commit()
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

What the code means

- `{reading_id}` in the path becomes the `reading_id: uuid.UUID` parameter; a malformed id is refused automatically.
- The `where` includes `HealthRecord.user_id == user.id`, so another person's reading is simply "not found" (ids cannot be probed).
- The score row is deleted first, then the reading; `AuditLog` records who deleted what, from which IP address.
- `204 No Content` means success with an empty body.

3. `frontend/src/lib/params.ts` · intParam and oneOf

```
 8  export function intParam(raw: string | null, fallback: number, min: number, max: number): number {
 9    if (raw === null || !/^\d{1,6}$/.test(raw)) return fallback;
10    return Math.min(max, Math.max(min, Number(raw)));
11  }
12
13  export function oneOf<T extends string>(raw: string | null, allowed: readonly T[], fallback: T): T;
14  export function oneOf<T extends string>(raw: string | null, allowed: readonly T[], fallback: null): T | null
        ;
15  export function oneOf<T extends string>(raw: string | null, allowed: readonly T[], fallback: T | null): T |
        null {
16    return raw !== null && (allowed as readonly string[]).includes(raw) ? (raw as T) : fallback;
17  }
18
19  export function textParam(raw: string | null, maxLength: number): string {
```

What the code means

- Anything in the URL is untrusted. `intParam` accepts 1 to 6 digits only (`/^\d{1,6}$/`) and clamps the value between `min` and `max`.
- `oneOf` returns the value only if it is in the allowed list; otherwise the fallback. The first two lines are **overloads**: they tell TypeScript the result can be null only when the fallback is null.

4. `frontend/src/pages/citizen/History.tsx` · History (state and delete)

```
13  export function History() {
14    const { params, set } = useQueryState();
15    const { data: info } = useResource(() => getMetrics(), "metrics");
16    const metricKeys = (info?.metrics ?? []).map((m) => m.key);
17
18    // Clamped and validated: nothing unexpected from the URL reaches the API.
19    const window = oneOf(params.get("days"), WINDOWS, "90");
20    const metric = info ? oneOf(params.get("metric"), metricKeys, null) : null;
21    const flaggedOnly = params.get("flagged") === "1";
22    const page = intParam(params.get("page"), 1, 1, 10_000);
23    const query = { page, page_size: PAGE_SIZE, metric, days: window === "all" ? null : Number(window), flagge
          d_only: flaggedOnly };
24
25    const { data, error, loading, reload } = useResource((signal) => getHistory(query, signal), JSON.stringify
          (query));
26    const [confirming, setConfirming] = useState<string | null>(null);
27    const [deleting, setDeleting] = useState(false);
28    const [actionError, setActionError] = useState<string | null>(null);
29
30    const remove = async (id: string) => {
31      setDeleting(true);
32      setActionError(null);
33      try {
34        await deleteReading(id);
35        setConfirming(null);
36        reload();
37      } catch (e) {
38        setActionError(errorMessage(e));
39      } finally {
40        setDeleting(false);
41      }
42    };
43
44    const pages = data ? Math.max(1, Math.ceil(data.total / PAGE_SIZE)) : 1;
45    const byKey = new Map((info?.metrics ?? []).map((m) => [m.key, m]));
46    const selectCls = "min-h-11 rounded-xl border border-line bg-surface px-3 text-sm";
```

What the code means

- `oneOf(params.get("days"), WINDOWS, "90")` reads `?days=` and falls back to 90.
- `useResource(..., JSON.stringify(query))`: the key is the whole query as text, so changing any filter loads again.
- `const [confirming, setConfirming] = useState<string | null>(null)` remembers which row is waiting for "Delete / Keep".

- `remove` is `async`: it sets `deleting` (the button shows a spinner), awaits the request, then `reload()`s; `finally` always clears the busy state.
- `Math.ceil(data.total / PAGE_SIZE)` is the number of pages.

Syntax used in this feature

Code	What it means
`Query(1, ge=1, le=10_000)`	A query parameter with limits.
`query.subquery()`	Use a query inside another one (here, to count it).
`.offset(n).limit(m)`	Skip n rows, then take m.
`JSON.stringify(obj)`	Turn an object into text.
`try { } catch { } finally { }`	finally runs whether the request worked or not.